PHY 3902: Physics and Applied Physics Laboratory I

Lab 1: Introduction to Data Acquisition with Arduino

**** This introductory experiment is to be completed by everyone individually. You may work in pairs to discuss the tasks, but everyone should write their own code and complete all tasks.**
**** EVALUATION: You do not need to submit a formal lab report for Lab 1. However, you will be asked by your TA to demonstrate certain capabilities to earn marks. You will also be asked to submit some of your code.**

Objectives:

- Learn how to use the Arduino board
- Understand the Arduino IDE development platform and its programming language
- Introduce oneself to data acquisition and instrument control using Arduino

Caution: ***electronic devices not connected to ground are very sensitive to static electricity, and can be rendered completely and irreversibly inoperative following a static discharge. Avoid working in dry carpeted rooms and discharge yourself regularly by touching a massive metallic object, copper water pipe etc.***

INTRODUCTION:

When designing an experiment to study a physical phenomenon, we generally want to measure something under different conditions. The measurements can be done manually but automation using a data acquisition (DAQ) system is often more accurate, faster, and can provide access to more information. The experimental conditions can also be changed manually or using a computer. In order to accomplish this, we can use the computer (a DAQ system) to INPUT voltages (to measure a signal) and to OUTPUT voltages (to change the experimental conditions). Of course, when designing experiments, you might want to acquire different types of signals (e.g., voltage, current, temperature, light intensity, etc.), but from a practical point of view, it all comes down to measuring voltages. Similarly, to "change the experimental conditions" (e.g., change temperature, move an object, activate an actuator, apply a voltage to a circuit, etc.), it all comes down to outputting voltages.

When using a computer to control an experiment, we typically need some hardware to act as an intermediate between the computer and the experiment. This is often called DAQ (data acquisition) hardware. There are different types of DAQ hardware, but they all allow you to INPUT voltages and OUTPUT voltages. However, different DAQ systems might be optimized for different types of experiments. But strictly speaking, they will differ by their resolution and speed, and the number of signals (i.e., channels) they are able to handle simultaneously.

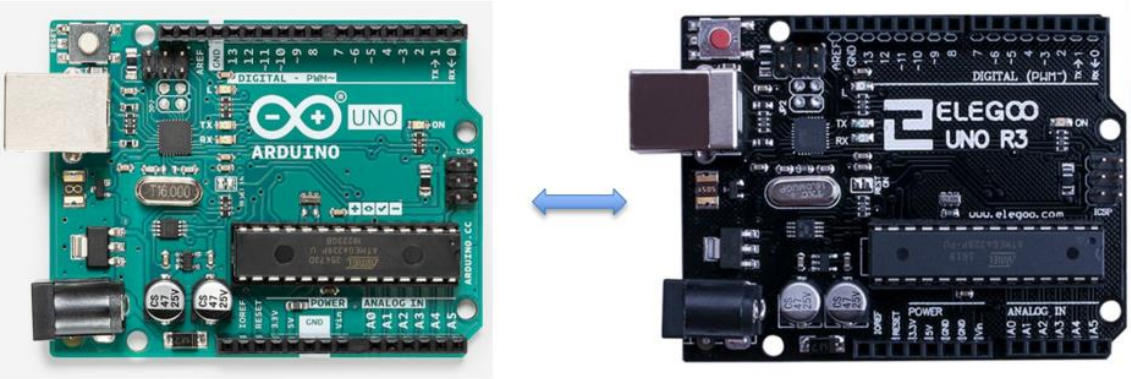
In our research laboratories, DAQ systems might have 16-bit resolution. What does this mean?! If the hardware is capable of handling voltages between 0V and 5V, then the smallest change in voltage it can discriminate will be $\frac{5V}{2^{16}} = \frac{5V}{65536} = 0.0000762V$. That's pretty good! The DAQ systems we typically use are pretty fast as well, capable of inputting or outputting voltages at 1 MHz. That's rather fast! However, these DAQ systems cost thousand$ of dollar$! Fortunately, many experiments do not require this much resolution and speed. The Arduino board is a DAQ system with 10-bit resolution (smallest change in voltage that can be detected will be $\frac{5V}{2^{10}} = \frac{5V}{1024} = 0.00488V$) and a bandwidth of around 10 kHz. This is actually not too bad, especially given that an Arduino board costs about $20!

In this course, we will use the Arduino platform as our DAQ system. The Arduino might not be as performant as a high-end DAQ platform, but conceptually, it can do the same things! The concepts you will learn using the Arduino platform are translatable to other DAQ systems.

This first introductory "experiment" will not really be an actual experiment. The goal will be to introduce you to the Arduino hardware, the Arduino programming language and some basic DAQ input and output capabilities. You will use these skills in the next experiments!

THE HARDWARE – Arduino UNO R3!

Arduino is a manufacturer of open-source electronic components and free software. It specializes in the design and production of microcontrollers and application kits. In order to use the Arduino, you need to write code (a set of instructions) to tell the Arduino hardware what to do. The code will be written in plain text, but this code will be processed and sent to the Arduino hardware using software called the **I**ntegrated **D**evelopment **E**nvironment (Arduino IDE). We will be using the Arduino UNO R3, however the following information can easily be adapted for other board versions. By the way, many companies make "Arduino boards" since the design is open source. We will be using "Arduino boards" manufactured by several different companies, including ones made by the company "Elegoo", but everything about these boards is pretty much identical to the one made by the company "Arduino", except that it is a little cheaper!



One of the things I like about the Arduino board is that not much is hidden. You can see all the electrical components and it's pretty clear that there are various connectors (pins) that are meant for you to connect wires to. The main connector is USB. You will connect your computer to the board using a USB cable. You will be programming some commands on your computer, and these commands will be transferred to the Arduino board by USB. Also, this USB connection provides power to the board. More on this later.

Here is some information about the other pins:

Power pins:

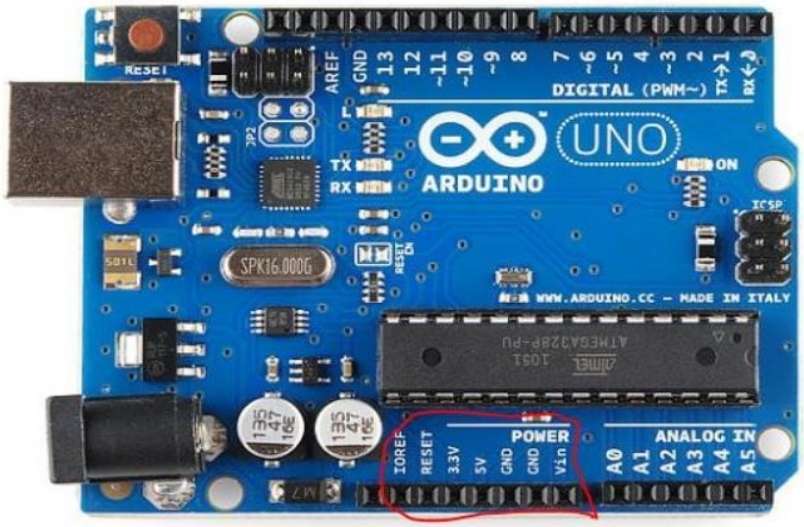


Figure 1: Arduino Board - Power pins

Some basic power pins are circled in red in figure 1.
To power simple electronic circuits, the board can provide constant voltages such as 5 V and 3.3 V from the indicated pins. There is also a **GND** ground pin for 0 V. The pin labeled **Reset** is rarely used but it can be used to reinitialize the board externally, and the button labeled **Reset** (top left of the board) allows you to perform this function manually. Pin $\mathbf{V_{in}}$ can be used to power the Arduino board using an external power supply if the USB connection is not used. The board has a voltage regulator, which can protect it when powered from 7 to 12 V supplies.

Analog (INPUT) pins:

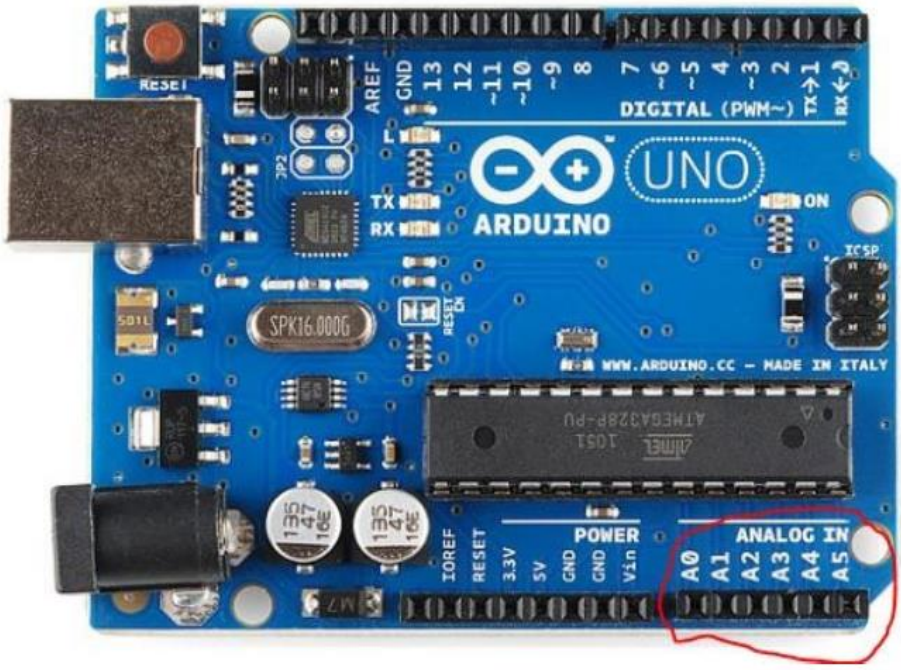


Figure 2: Arduino Board: Analog pins

Analog pins are used to read "non-binary values" as produced by sensors. In other words, the Arduino can measure voltages of wires connected to these pins. There are 6 pins, so you can measure up to 6 different analog voltages. The board features an analog-to-digital converter (ADC) which can convert an analog voltage into a digital number that can be used by the microcontroller and processed by the program running on the Arduino. This digital number will be an integer between 0 and 1023, since this is a 10-bit measurement. The voltage range you can measure is 0V to 5V. For example, if your code asked to measure the voltage on a certain analog input pin and it returns a value of 645, this corresponds to a measured analog voltage of 3.152 V. This is calculated as $\frac{645}{1023} * 5\ V = 3.152\ V$.

Digital (OUTPUT or INPUT) pins:

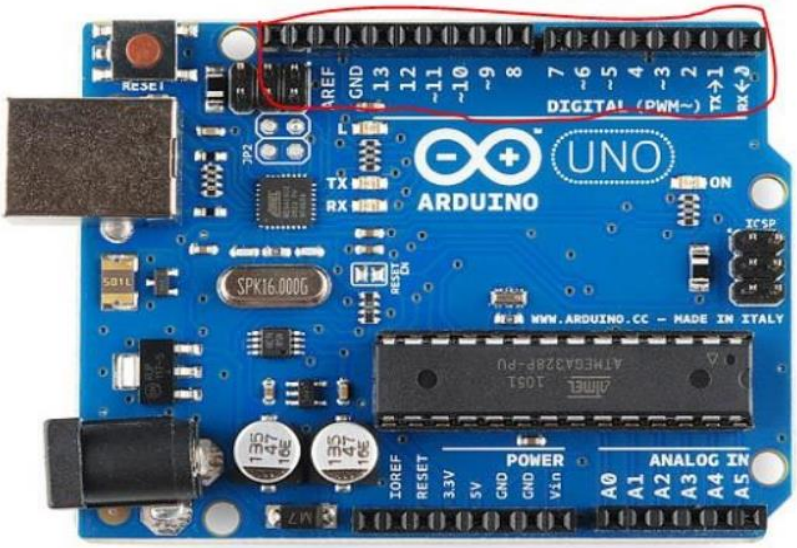


Figure 3: Arduino Board - Digital pins

Digital pins (Figure 3) are used for binary logic (1 or 0) input or output. For instance, a sensor connected to one such pin needs to provide either one of two voltages: **High** or 5 V when it is triggered, otherwise **Low** or 0 V when idle. The digital pins can also be used as outputs and provide either 5 V or 0 V to the device connected to it. The board also features a light-emitting diode (LED) under the named pin 13, which can be used as a kind of indicator during operation. This LED is protected by a series resistor.

*** Special comment on powering an LED using these digital pins:

Caution: never connect an LED directly to any board pin! *Voltages present (5 V or 3.3 V) are sufficient to completely destroy the LED in a fraction of a second****. One must always place a resistor in series with the LED to limit the current.*** *During operation, it's recommended to keep the voltage across the terminals of the LED between 1.8 V and 3.6 V. We will consider 1.8 V to make sure that the value of the resistor we choose is enough to prevent the LED from burning. One must choose the resistor to limit the current to approximately 20 mA. For instance, if one wants to connect one such diode to a 5 V supply, the resistor will need to drop (5 V – 1.8 V) = 3.2 V. The required resistor value will thus be 3.2 V / 0.020 A = 160 Ω. Any resistor at least equal to this value will do. I recommend >200 Ω to be safe.*

Using the Arduino – programming

Using the Arduino involves writing some code on your computer and transferring these commands to the Arduino board. In principle, once you transfer your code to the board, the Arduino does not need to maintain a connection to the computer in order for it to work. But you do need to provide power! This is rather awesome as it means you can program your Arduino to run as a stand-alone system using a battery to provide power. In our experiments, you will always keep your computer connected to the board with the USB cable. This will provide power to the board and allow you to monitor your measurements in real-time.

An Arduino program (code) is called a ***sketch***. You can write your sketch using software called Arduino IDE. Arduino IDE, the programming tool for the board, separates these programs in three main parts.

- The first part is used to declare variables and function libraries that will be used.

- Then, the function **setup()** is executed only once at reset, and is used to initialize input and output pins, USB communications protocols, etc.

- Finally, the actual instructions you want the Arduino to run are included in the function **loop()**, which is executed completely and then repeated again and again, non-stop, until the power is turned off (i.e. the USB cable is disconnected) or you press the reset button on the Arduino itself.

Details on installing the Arduino IDE software on your computer are added as an appendix to this document. If you are using a lab computer, Arduino IDE should be installed already.

The appendix also highlights a set of commonly used functions. You can use this as a reference when you are writing your sketches. Do not hesitate to use the internet to find information on other functions. However, a good way to learn the Arduino language is to edit a sketch that already exists. In this introductory experiment, you will be given some basic sketches that include comments. Make sure you understand every line of code in these sketches!

The "experiment"

1: Outputting a voltage – Part 1 (to light an external LED)

You can program the Arduino to output voltages. As discussed above, the Arduino is only able to output a "digital" voltage, which means it can output 0V or 5V, but nothing in between. This is somewhat disappointing, but we will improve upon this later!

For this first part, we will use a built-in example (**BLINK.ino**) to highlight a few basic concepts. A step-by-step guide is included in the APPENDIX. You should perform those steps now if you have never used an Arduino. Here, we only describe the main ideas.

1.1. DIGITAL OUTPUT

The sketch file **BLINK.ino** (see Appendix) is a simple program that uses the function **digitalWrite(13, HIGH)** to output a voltage from pin 13. Since pin 13 is permanently connected to an LED on the Arduino board, you should see the LED blinking! Try changing the delay to make it blink faster.

Of course, you can also use a wire to connect this output pin to an external LED mounted on a breadboard. Try this yourself, but make sure to add a resistor in series (ideally ≥ 200 Ω) with the LED, in order to limit the current (as discussed above).

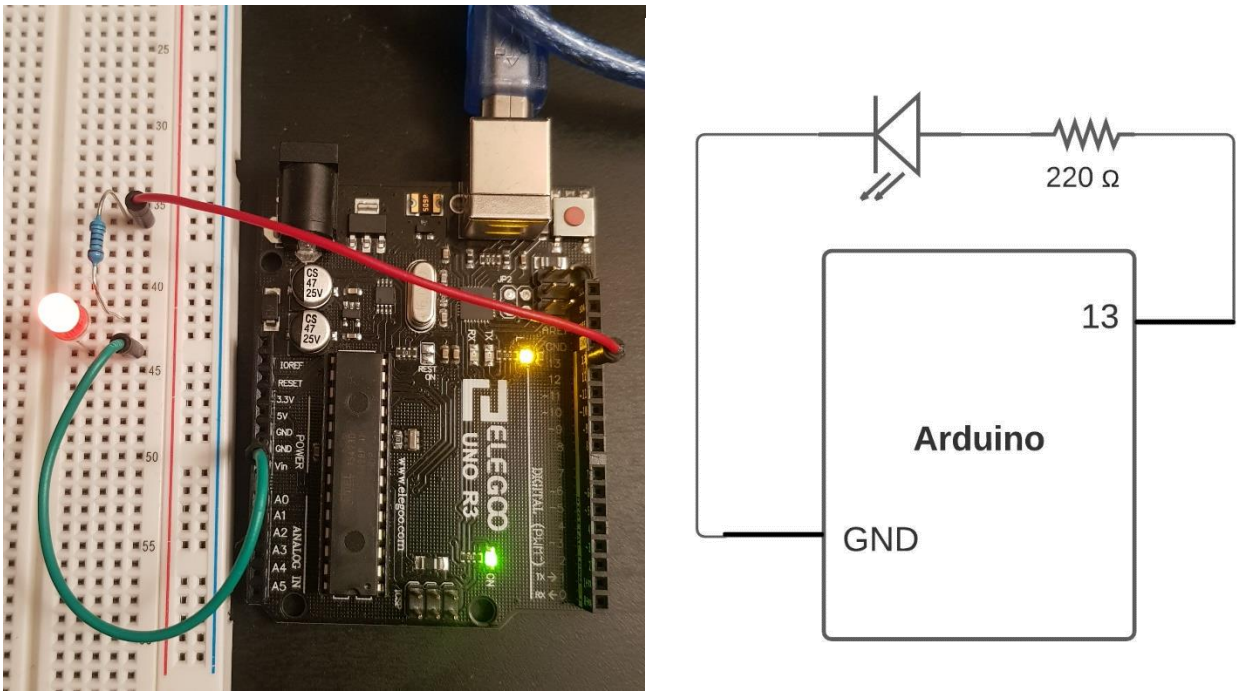


Figure 4: Circuit diagram of a blinking LED

This would work with any of the digital output pins (1 to 13), of course. You can also light multiple LEDs independently using multiple digital output pins simultaneously. You could even program your own light show! Of course, you can imagine outputting a voltage for reasons other than lighting an LED, depending on the experiment.

1.2. PWM (PULSE WIDTH MODULATION) VERSUS TRUE ANALOG OUTPUT

Outputting digital voltages (0 V or 5 V) is nice but is rather limiting. It would be nice to be able to output any voltage value between 0 V and 5 V. Many more advanced DAQ hardware are capable of outputting true analog output voltages at high resolution, but this capability adds to the cost of the hardware. The Arduino is not capable of doing so on its own, but it can use pulse width modulation (PWM) to accomplish something similar.

PWM (*pulse width modulation*) is a method for converting a digital signal of fixed voltage (say, 5 V) into a quasi-analog signal between 0 V and 5 V by rapidly switching this fixed voltage. The average value will depend on the duty-cycle of this signal. If the voltage is always ON, the average value will be maximum. If the cycle is 50% ON, 50% OFF, the average value will be half of the maximum etc. This method is useful for controlling, for instance, the light intensity of an LED or a heating element which has a certain inertia and only reacts to the average value of the voltage anyway. In order to control a faster-responding device, the frequency of this voltage flashing might become apparent and

cause problems, and in this case one would need to produce the analog voltage another way: using a digital-to-analog converter (DAC).

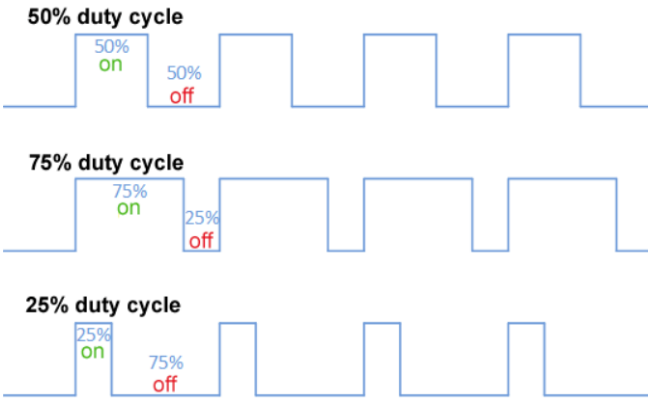


Figure 5: Duty cycle

On the Arduino, pins capable of outputting a PWM signal are digital ones labeled with symbol tilde '~'. For digital pins, including those used for PWM, the values that can be used only have 8-bit resolution and thus vary between 0 and 255, which will be translated to an average voltage between 0 V and 5 V respectively, divided into 2^8 or 256 different levels. On the other hand, pins used for analog input are connected to a 10-bit analog-digital converter (ADC), resulting values can thus vary between 0 and 1023 (i.e. 10-bit resolution), this corresponding again to 0 V and 5V, respectively. For more information: *https://www.arduino.cc/en/tutorial/PWM*

Task:

The sketch file called **LED_FADE.ino** is a basic example of using PWM outputs to control the intensity of light emitted by an LED. This sketch uses the function **analogWrite(9, value)** to output a PWM signal from the pin number 9. You could change the output pin to any of the digital pins that have the '~' sign. The variable "value" can be any integer from 0 to 255, which correspond to duty cycles going from 0% to 100%. For instance, a value of 64 would correspond to approximately 25% $\left(\frac{64}{255} \times 100\right)$ duty cycle.

- Connect pin 9 to a ~200 Ω resistor placed in series with an LED, and complete the circuit to ground. Make sure the positive side of the LED, the anode, which is the longer lead, is connected to the resistor, and the shorter lead to ground. This is shown in Figure 6.

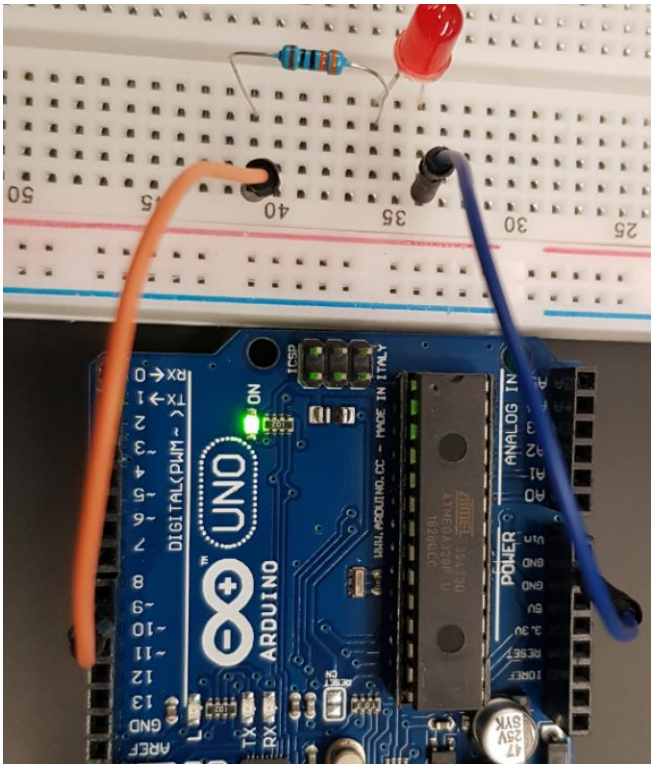


Figure 6: LED in series with a resistor to limit the current.

- Read the comments in the sketch file and make sure you understand every line of code, as you will need to use similar functions in future experiments.
- The code includes some “FOR loops” that are used to increment the output voltage up or down by specified increments.
- Upload the sketch to the Arduino and observe that the LED intensity fades on and off.
- Of course, feel free to change the code to play around with this behaviour!

While PWM can be quite useful, some experiments might require that you output a true analog voltage between 0V and 5V. You will need to do this in a future experiment (Current versus Voltage characteristics of LEDs), and we will use a little circuit to accomplish this. More details will be explained then.

2: INPUTTING VOLTAGES – measuring temperature with a thermistor or light intensity with an LDR

A thermistor is a type of resistor whose resistance is highly dependent on temperature. Measuring temperatures is very important in many experiments. There are a few different types of sensors used to measure temperatures (e.g., thermocouples, infrared sensors, bimetallic devices, etc.) but we will focus on thermistors here.

TASKS:

Since thermistors are resistors, we will build a simple voltage divider circuit, as highlighted in the following circuit diagram:

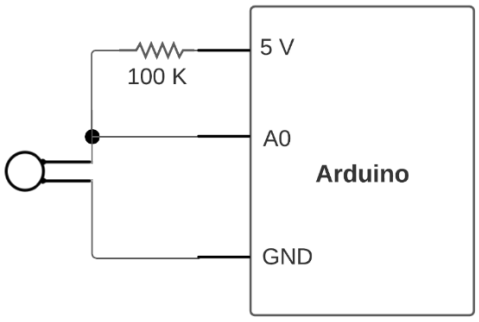


Figure 7: Thermistor voltage divider

It is best to choose a series resistor value similar to that of the thermistor. I suggest using 100 kΩ, but you can choose any value that's close to the thermistor's resistance.

Use one of the 5V output pins of the Arduino as your constant voltage source. Connect the ground to one of the ground pins of the Arduino. Use one of the analog input pins (A0 to A5) to measure the voltage between the resistor and the thermistor.

- Upload the sketch named "**THERMISTOR.ino**" to your Arduino. Read the included comments carefully to understand each line of code.

- The function **analogRead(A0)** will return the voltage measured on pin A0. This function will return an integer value between 0 and 1023, since this pin operates at 10-bit. You can use the formula $\boldsymbol{5V * analogRead(A0)/1023}$ to get the actual measured voltage.

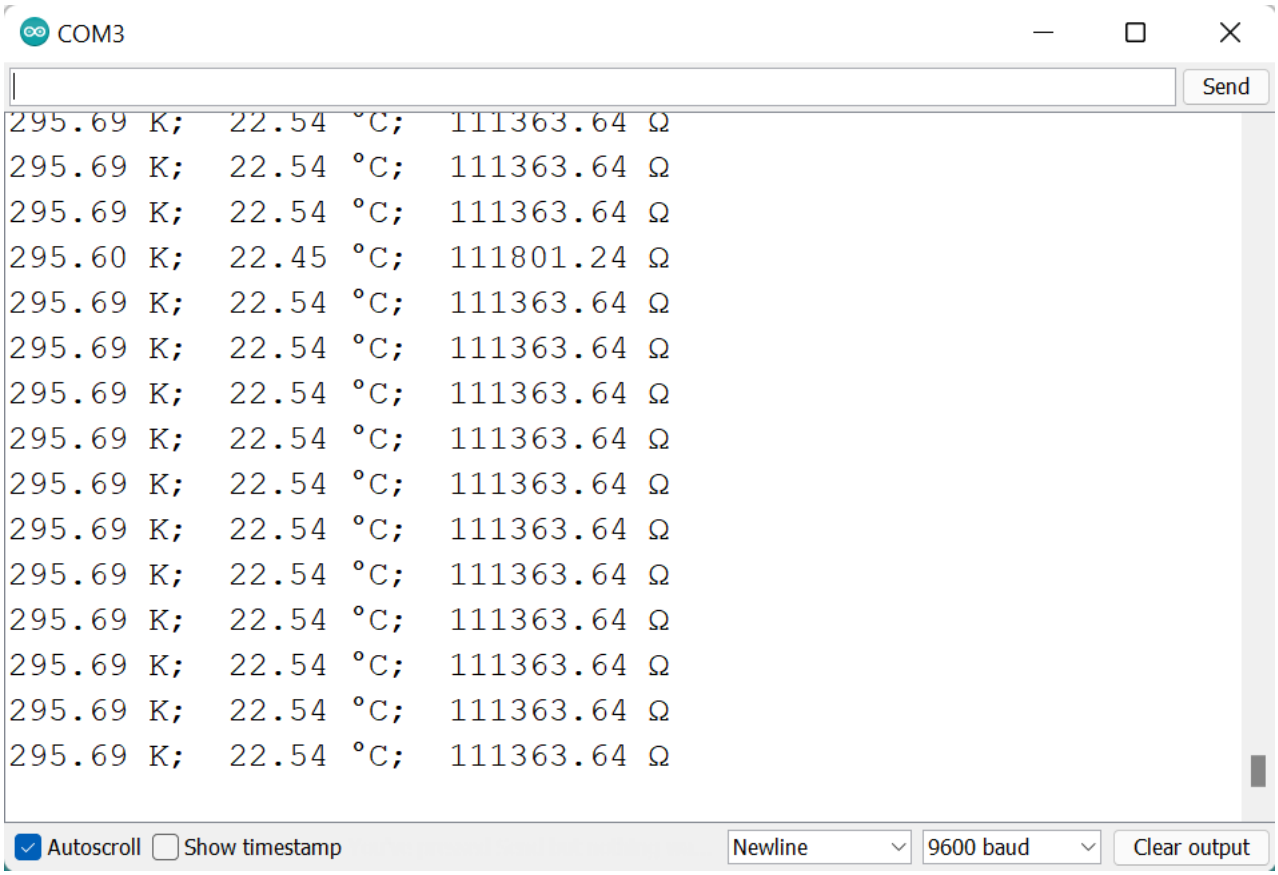
- If you remember last year's electronics course, you should be able to figure out a way to calculate the resistance value of the thermistor by knowing the measured voltage and the value of the series resistor! The sketch already calculates the thermistor's resistance $\mathbf{R_t}$.

- The sketch uses a few "Serial" functions. The ones in **setup()** are initialization functions. The ones in **loop()** are used to send some values to the serial monitor. Click on the menu **Tools > Serial Monitor** to open the serial monitor window. You should see some values being displayed every once in a while.

- In order to convert the measured thermistor resistance to an actual temperature, you must use the Steinhart-Hart equation. The Steinhart-Hart equation requires some calibration constants specific to the thermistor you are using (A = 0.7141095422E-3, B = 2.143243110E-4, C = 1.129790692E-7). These will be declared in your sketch. Modify your sketch to convert the thermistor resistance to an actual temperature (in Celsius and Kelvin). This equation is included below:

```
Vo = analogRead(A0); // read analog to digital converter value
Rt = R0 * (Vo / (Vin - Vo));
lnRt = log(Rt);
Tk = (1.0 / (A + B*lnRt + C*pow(lnRt, 3)));
Tc = Tk - 273.15;
```

- Modify the serial output so that it refreshes every second and looks like this, and **show this to your TA**:



- So…… what's the lab temperature?! How would you calibrate your thermistor to make sure your measurement is accurate?
- Replace the thermistor with a light-dependent resistor LDR (Light Detection Resistor). It is recommended that you also change the value of the series resistor so it's close to that of the LDR (about 2-10 kOhm). The resistance of the LDR is proportional to the light intensity detected by the sensor. The measured voltage is now proportional to the light intensity! You can use the same voltage divider concepts as you did with the thermistor.
- ***As a challenge, write a new sketch that uses the LDR to measure the light intensity and connect one of the PWM pins to power an external LED. This will become your first code that operates in feedback. As you wave your hand over the LDR to block the light impinging on the LDR, the LED should become brighter or darker. Try to ensure that the LED brightness decreases when you block the light to the LDR. When you accomplish this, show this to your TA.***

3: Outputting Voltages - Part 2 – controlling motors!

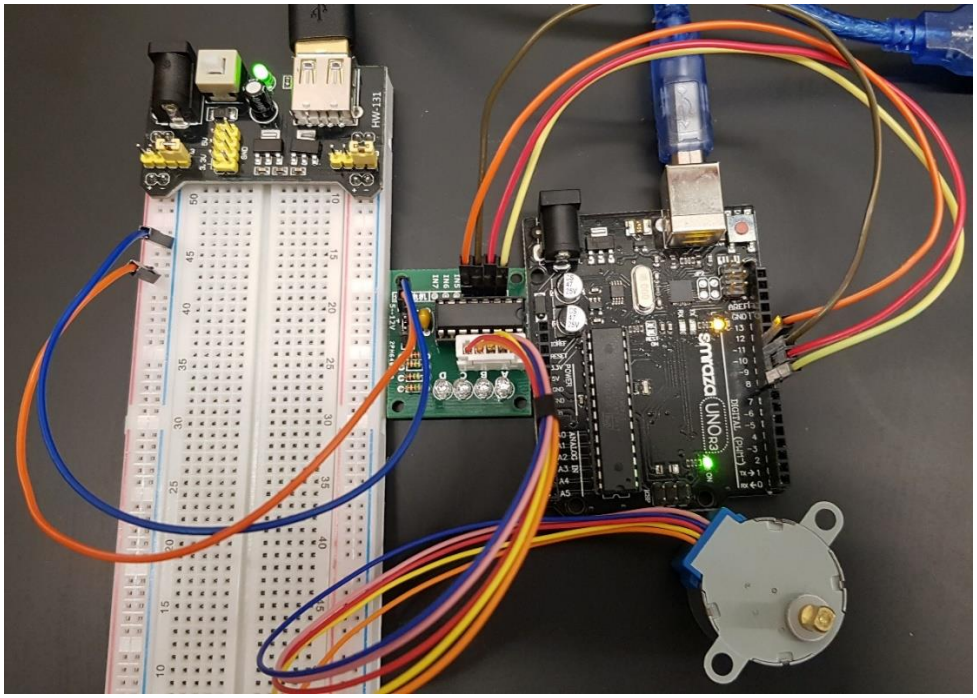
Outputting voltages is useful when you want to control systems in an experiment. Here, we will learn how to control two types of motors: stepper motors and servo motors. Both types of motors have advantages and disadvantages, though we are not going to get into the details here. You can certainly read about the differences online. But, servos are usually quicker and have higher torque. The electronics and coding for stepper motors is slightly more complex. Each motor type has its own dedicated "motor driver" circuit board. These motor driver cards will be connected between the Arduino and the motors. Fortunately, the motors we will use have pre-written software libraries that will make using them straightforward. It should be noted that motors can require a little more power than the Arduino is capable of supplying. For this reason, we will use separate breadboard-mounted power supply to power the motors. You will use these motors in future experiments.

TASKS

The following tasks are not particularly challenging, as you simply need to run the given sketches. But you should notice that the code includes libraries specific to each motor. These libraries (declared at the beginning of the sketch) can be added using the menu **"SKETCH>INCLUDE LIBRARY"**. We will be using standard libraries that are bundled with Arduino IDE, but you can upload other libraries if required. In any case, make sure you are able to control these motors (angle of rotation, speed, direction).

3.1 Stepper motors

- The sketch "**STEPPER.ino**" is provided to allow you to rotate the stepper motor.
- Wire up the motor as in the following circuit diagram:



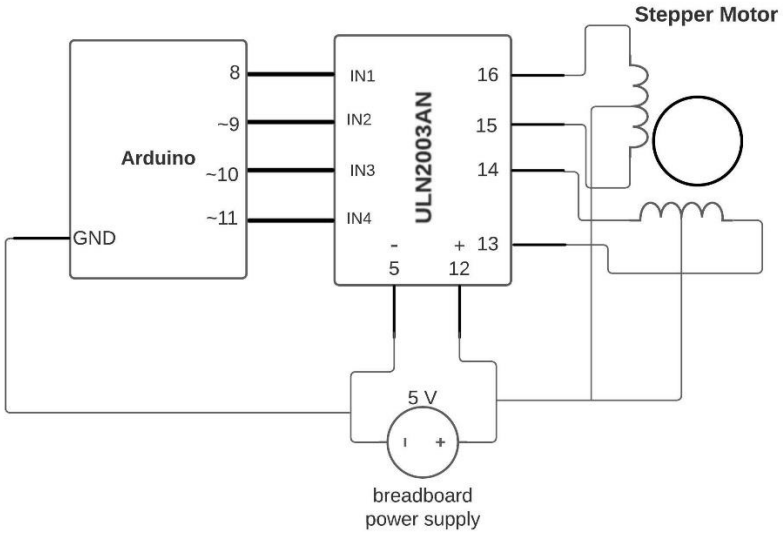


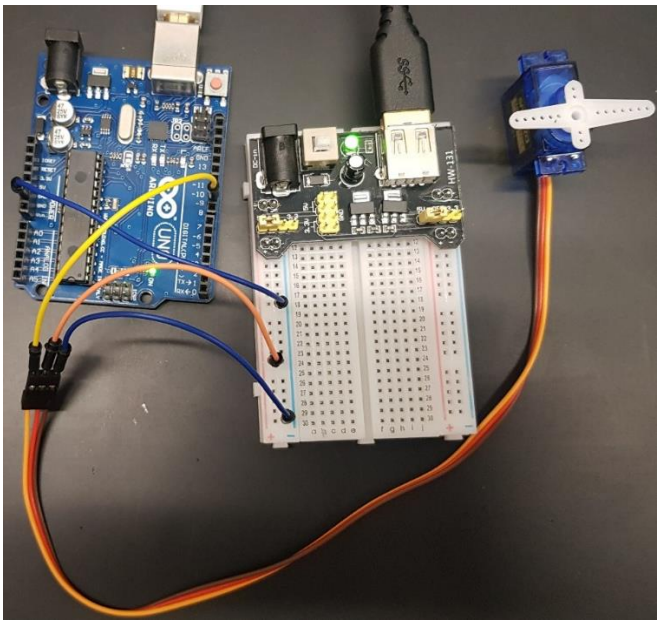
Figure 8: Stepper motor circuit and diagram

- As mentioned above, the motor "driver circuit" is powered by a little power supply mounted on the breadboard. Make sure to use this power supply, instead of using one of the Arduino output pins for power, since the Arduino cannot supply enough power without a little extra help! You can power the power supply using a USB cable connected to your computer or using an external power supply.
- CALIBRATION: the command used to rotate the stepper motor requires us to tell it how many steps to rotate. Unfortunately for you, I am a little old and cannot remember how many steps are required to rotate the stepper motor by 360 degrees. Determine a method to calibrate the stepper motor so that you can rotate it by an accurate angle value. This will become VERY important in a future experiment.

3.2 Servo motors

Servo motors are nice as they move a little faster than steppers. Also, you can keep track of the position and rotate them with respect to an initial position.

- The sketch "**SERVO.ino**" is used to rotate the Servo motor.
- Wire up the servo motor as shown in the following circuit diagram.
- CALIBRATION: Use a similar method as above to calibrate the rotation of the servo motor.



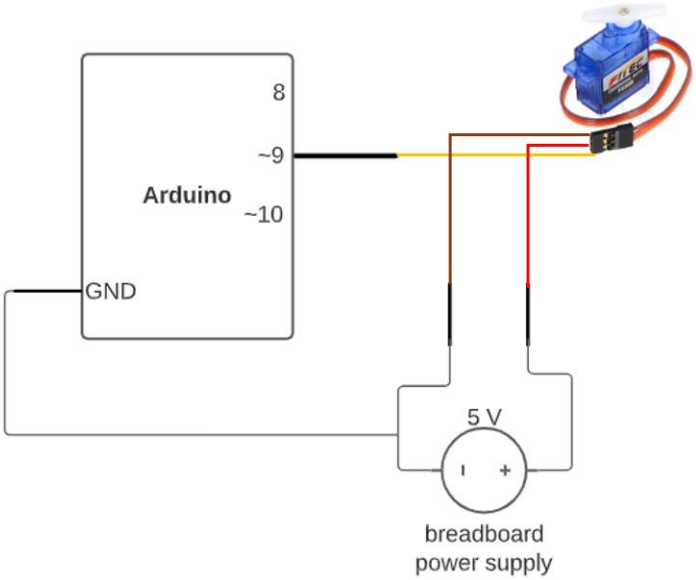


Figure9: Servo motor setup and circuit diagram

4: Final challenge – Google's Dinosaur game!

In follow-up experiments, we will use the above input and output capabilities to perform some simple experiments. Many science experiments involve making measurements (input) while changing experimental conditions (output). This can even be done in feedback, where the output depends on the input value.

For fun, I challenge you to write a sketch that can automatically play Google's dinosaur game. As you might know, if you go to https://www.google.com while you are disconnected from the internet (or type "**chrome://network-error/-106**" as a URL into Google Chrome, even when online), a little dinosaur appears. On the lab computers, I suggest you temporarily disable the network adaptor. Ask for help if necessary.



No internet

Try:

- Checking the network cables, modem, and router
- Reconnecting to Wi-Fi
- Running Windows Network Diagnostics

ERR_INTERNET_DISCONNECTED

If you press on the space bar (or arrow key), the dinosaur starts moving and the game begins. Every time you press on the spacebar (or arrows), the dinosaur jumps. The object of the game is to avoid the various obstacles (cacti and birds). My high score is 3361.



No internet

Try:

- Checking the network cables, modem, and router
- Reconnecting to Wi-Fi
- Running Windows Network Diagnostics

ERR_INTERNET_DISCONNECTED

TASKS:

- Use the servo motor and program the Arduino to press on the spacebar. You can use tape, reusable adhesive, or anything to secure the motor on your keyboard.

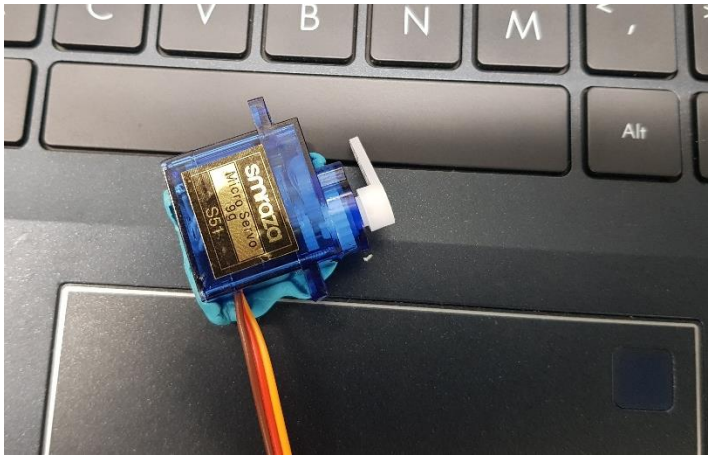


Figure 10: Secured servo motor

- Use the LDR light sensor to detect the incoming obstacles. Again, you can use anything to secure the LDR light sensor to the computer screen.

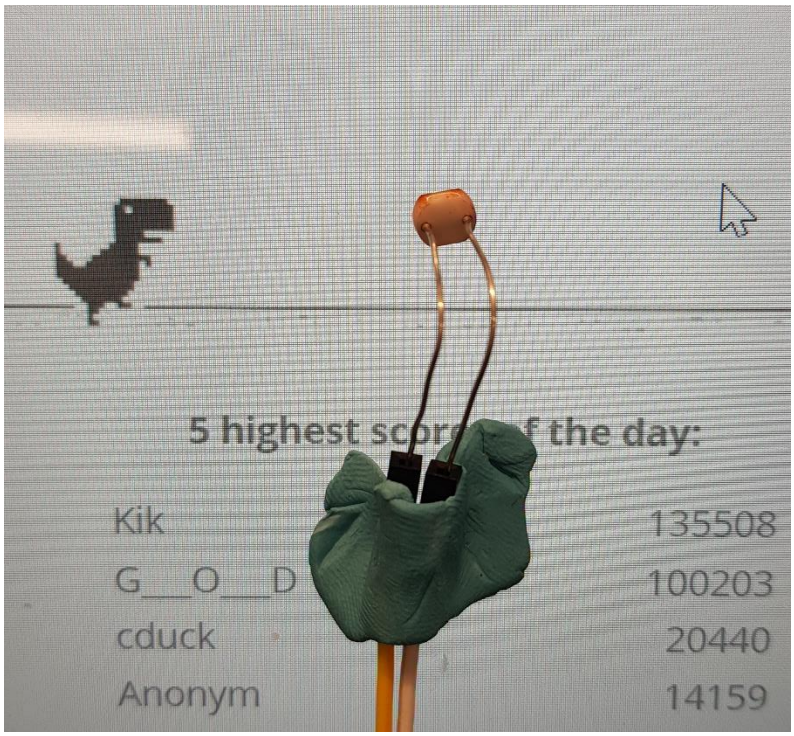


Figure 11: Secured LDR

- Combine the LDR sensing code and the servo motor code into a single sketch file in order to have the Arduino control the dinosaur to jump at the right moment after the LDR detects an obstacle.
- The sketch "**DINOSAUR.ino**" provides the basic functionality that you need. But you will need to optimize it… a lot!

Notes:

- The cacti and birds will move at increasing speeds as the game progresses. You will need to change the delay between the LDR detection and the servo motor actuation as the game progresses. ***Add a comment to your code explaining your strategy.***

- The LDR light intensity threshold (difference between the white screen and the dark obstacles) will have to be determined. This will be different for each setup. ***Add a comment to your code explaining your strategy.***
- The dinosaur can also "duck" below some birds. You might have to deal with that!
- Perhaps the biggest challenge is when the screen turns dark, and the obstacles becomes white. This will require you to change the criteria you use to trigger a keyboard press. The example code does not deal with this challenge very well. Propose a way to deal with this situation. ***Add a comment to your code explaining your strategy.***
- **Show your TA how well your Arduino is able to play this game!** Try to get to the dark screen at least. You can also take a video as "proof"! I'm curious what the highest score can be!
- **Submit your sketch to Brightspace.** Make sure to add lots of comments, including ones that explain how you dealt with the increasing speed and with the change in background colour.
- **Can you think of other simple online games that could be played automatically by the Arduino?** Please let me know!

APPENDIX

Downloading Arduino IDE:

For this introductory experiment, the Arduino IDE software and its preinstalled libraries of functions is all we will need. However, different sensors and actuators may use specific libraries. In order to use those specific libraries, the user needs to download the relevant material from the Internet and include it in Arduino IDE to access these functions. The Arduino IDE can easily be installed on your computer, and the link to obtain the version suited to your computer is: *https://www.arduino.cc/en/main/software*

Note that you can change the software language (English, French...) after installation via section **File > Preferences**. The above website also contains a large quantity of information about the Arduino board itself (*Hardware*), about the IDE etc. and it is strongly suggested to explore the site.

If one needs to use an external device connected to the Arduino board, the steps needed to install the required libraries are: if the downloading link is not found in the instruction manual of the device, an Internet search should find the required file; download this material in .zip format. Then in the Arduino IDE, click on **Sketch > Include library > Add .ZIP library...** Once these steps are completed, a new window should pop open where you will need to select the zip file you just downloaded and click **Open**. Arduino IDE will take care of installing those libraries and should return a success message at the bottom of the window. These steps are to be done every time a library for a new device needs to be used. For more information, the Arduino website has guides for all of this: *https://www.arduino.cc/en/guide/libraries*

Arduino IDE software and programming language:

One specifies what an Arduino board does via a program (*sketch*) written inside Arduino IDE then uploaded to the device using USB. The language used to program the Arduino is simple and based on C/C++. This language is easily learnt via examples, but here are some instructions and their syntax to help beginners and experimented users. Note that this language is case-sensitive! For more information, please visit the Arduino website and/or ask your TA!
https://www.arduino.cc/reference/en/ or *https://www.arduino.cc/reference/fr/*
(depending on your preferred language)

- **Comment "/* */" or "//":** This syntax prevents execution of part of your program or allows you to provide useful comments about your code. While "/* */" allows to comment-out what is between the asterisks, one can comment-out a whole line by placing "//" at the beginning of the line, preventing execution of the rest of the line.
- **Semicolon ";":** This informs the compiler of the end of a code line. The compiler's task is to convert the written program code into machine language directly understood by the Arduino chip.
- **Function "setup":** This function is used to initialize variables and determine communication parameters between the Arduino board and host computer such as bit rate etc. This function is

executed only once at reset or power up to set up necessary parameters. For more information, see page:
https://www.arduino.cc/reference/en/language/structure/sketch/setup/

- **Function "loop":** This function is the main part of the program and is executed repetitively and continuously as long as the board is powered or until the **Reset** button is pressed. The contents of the loop are executed from top to bottom, then it returns to the top. Consult page *https://www.arduino.cc/reference/en/language/structure/sketch/loop/*
- **Brackets"{}":** The parentheses delimit instructions in the desired way within the definition of a function. A function can be thought of a series of instructions delimited by brackets, and given a convenient name.
- **pinMode():** This instruction defines the operation of pins, either **INPUT** or **OUTPUT**, and syntax is: pinMode(pin number, mode)
- **delay():** This allows to pause the execution of the program a specified number of milliseconds.
- **digitalRead():** Reads input from a digital pin, either **High** or **Low**. The relevant pin must be declared as input in **setup**. Syntax is: **digitalRead(Pin number)**.
- **digitalWrite():** Writes, i.e. sets the voltage for a digital pin, either 5 V for **High** or 0 V for **Low**. The relevant pin must be declared as output in **setup**. Syntax is: **digitalWrite(Pin number, value)**.
- **analogRead():** Reads input voltage from an analog pin, returned value is between 0 and 1023 (10 bit resolution). The relevant pin must be declared as input in **setup**. Syntax is: **analogRead(Pin number)**.
- **serial.begin():** Initializes the frequency for USB communication in order to use the Arduino IDE software monitor to receive information from the board. Usually **serial.begin(9600)** for 9600 baud, about 9600 bits/sec. This needs to appear in **setup**.
- **serial.print():** To send variables or sentences to the monitor window of the Arduino IDE software, which can opened with **Tools > Serial monitor**. We will use this monitor to acquire data from the Arduino. The function **serial.println()** does the same thing but ensures that the next serial output ends up on a new line.

Prototyping board:

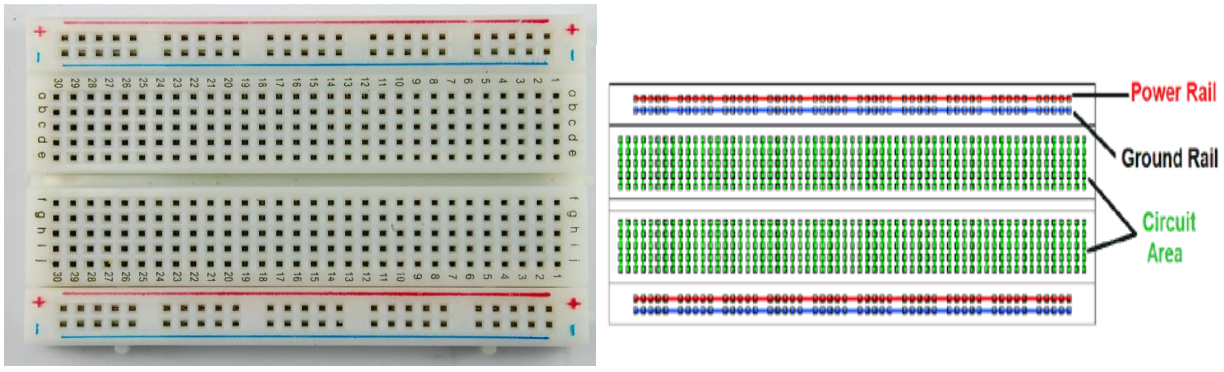


Figure 1: Explanation of prototyping board pins

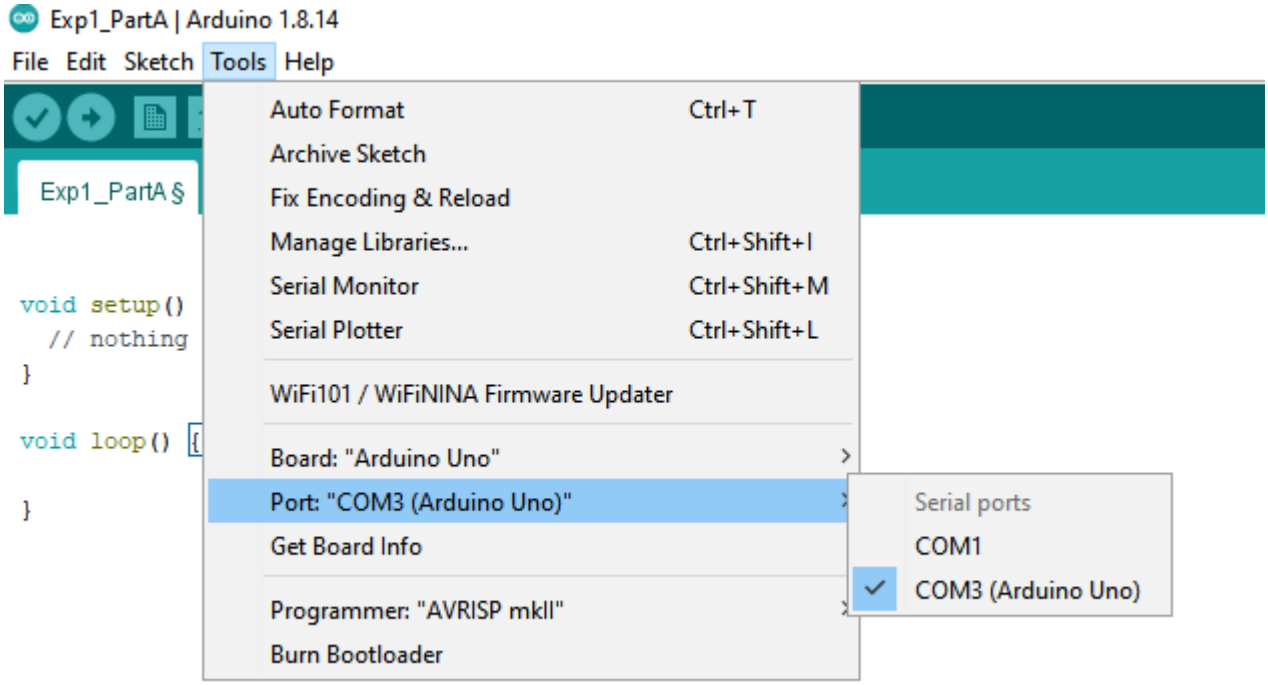
The above figure shows a typical prototyping board which is a convenient way of building circuits in a non-permanent way. It is divided in two sections: terminal columns and distribution rows. Terminal columns are in the middle and are connected vertically in the figure. The gap in the center separates the two sets of

terminal columns, and its width corresponds to the width of standard integrated circuits. The distribution rows allow access to positive supply (+) and ground (-) of the circuit. Prototyping boards sizes vary depending on needs.

DETAILED INSTRUCTIONS FOR PART A - Programming the integrated LED:

The following describes how to upload your code (sketch) to the Arduino board. It highlights a simple program that will make the integrated LED blink. For this, we will use a built-in example sketch called **Blink.ino**.

1. Connect the Arduino board using its USB port to your computer USB port. LEDs on the board should flash as the board is powered from the USB cable and is already running whatever program happens to be stored on it.
2. Launch Arduino IDE on your computer and select **Arduino Uno** on menu **Tools > Board:.** Then in **Tools**, select **Port** then select which USB port shows you Arduino board attached. Usually, it is called **COM** followed by a number. In case you do not see your board, unplug and re-plug it. If you are using the serial monitor to access the Arduino output, don't forget to use the same port number!



3. Open **File > Examples > 01.Basics > Blink**. This should open a new IDE window with the code:

```
  Turns an LED on for one second, then off for one second, repeatedly.

  Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO
  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to
  the correct LED pin independent of which board is used.
  If you want to know what pin the on-board LED is connected to on your Arduino
  model, check the Technical Specs of your board at:
  https://www.arduino.cc/en/Main/Products

  modified 8 May 2014
  by Scott Fitzgerald
  modified 2 Sep 2016
  by Arturo Guadalupi
  modified 8 Sep 2016
  by Colby Newman

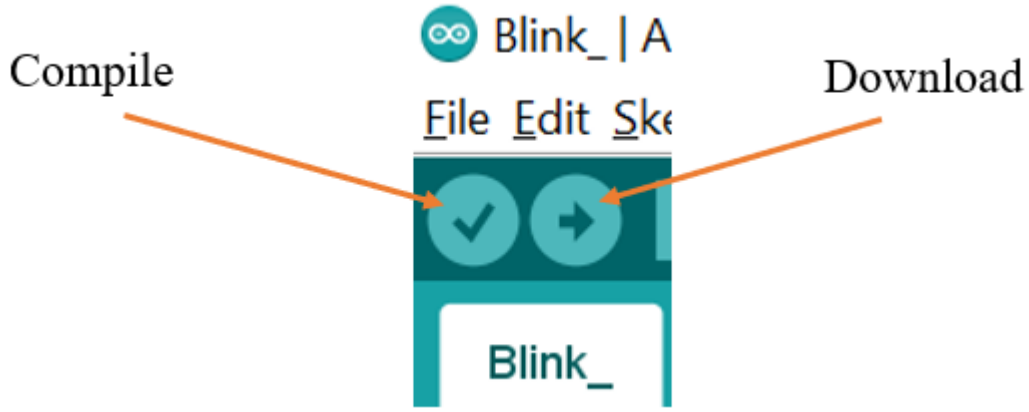
  This example code is in the public domain.

  http://www.arduino.cc/en/Tutorial/Blink
*/

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

4. Next step is to compile this code and upload the resulting program to your board. To compile your program, you need to click the big check mark on the top left. The compiler will try to make a machine language version of your program suitable for transfer to the Arduino—provided there is no error! If this is successful, the program can be uploaded to the board by clicking the large arrow. The code will be compiled again and if successful, will be uploaded to the Arduino board. If you know your code is error-free, only the big arrow step is required. The program begins to execute immediately after uploading.



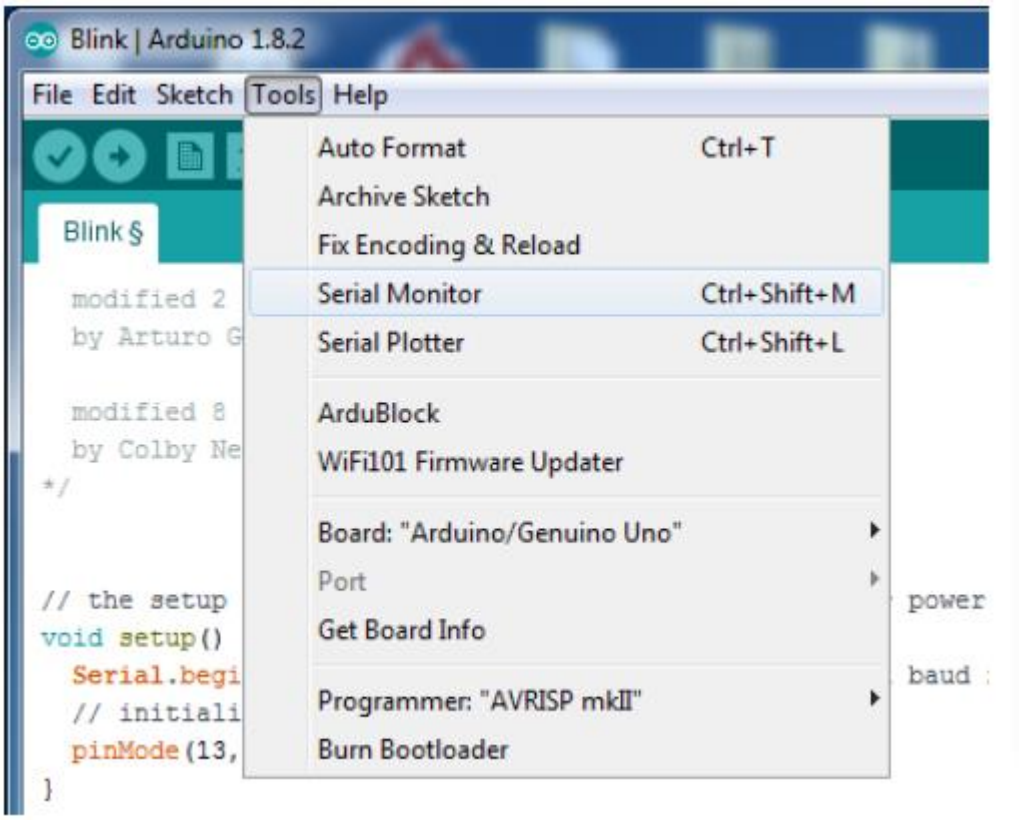
5. Once your program is uploaded, the LED labeled “L” should flash slowly. If you wish to restart your program, press the **Reset** button.
6. Try replacing constant **LED_BUILTIN** by the value 13, this should mean the same thing (in fact, the constant **LED_BUILTIN** is always defined to be equal to 13). You will now verify the value

corresponding to the LED by writing to the “serial monitor” of the Arduino IDE. You need to modify your program by adding **Serial.begin(9600)** in **setup**, which will initialize communication between Arduino and your computer. The actual writing is done by the command **Serial.print(LED_BUILTIN):**

```
// the setup function runs once when you press reset or power the board
void setup() {
  Serial.begin(9600); //start a serial connection for a baud rate of 9600
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(13, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
  Serial.print(LED_BUILTIN);        //print the value of LED_BUILTIN
}
```

7. Upload your new program.
8. To open the serial monitor, select **Tools > Serial monitor**.



What do you observe? At what rate is the information printed on the monitor? Is there another LED flashing now?

9. Replace **Serial.print(LED_BUILTIN)** by **Serial.println(LED_BUILTIN)**. Upload your program again and open the monitor. Things should be formatted a little better one line at a time.
10. Finally, add in **setup** at its end something like **Serial.println("Hello!!!")** (without forgetting the semicolon), and **Serial.println("On")**, **Serial.println("Off")** at the right places!

LED specifications:



Dimensions de l'article :

Diamètre de la tête : 5 mm.

Longueur de la broche : 18 mm.

Tension : rouge/jaune : 1,8 V-2,3 V.

Bleu/vert/blanc : 2,8-3,6 V.

Courant maximal : 20 mA.